NovruzCTF 2026

Technical Participation Report

1. Overview

Event: NovruzCTF 2026
Organizer: QarabagTeam
Date: March 28–29, 2026
Duration: 24 hours
Format: Jeopardy-style

NovruzCTF is an online cybersecurity competition covering multiple domains such as Web, Cryptography, Reverse Engineering, and Pwn. The event focuses on practical problem-solving and offensive security skills.

Statistics:

- Teams: 348
- Rank: 84
- Score: 2700

2. Team & Role

Team: m4ake_2
Members: 3

- red warrior (me)
- aboabdo
- mohammedjaly2010

My Contributions:

- Solved Web challenges
- Assisted with Python scripting
- Attempted Cryptography & Reverse Engineering

3. Web Challenges (Solved)

I solved several Web challenges by analyzing HTTP behavior and hidden endpoints.

Methodology:

- Intercepting requests via browser DevTools
- Analyzing endpoints and responses
- Identifying encoded data and decoding it
- Using logic instead of brute guessing

Tools:

- Browser DevTools
- Python (basic automation)
- Encoding tools (Base64, XOR)

4. RSA Challenge (Deep Technical Attempt)

- ### Challenge Description

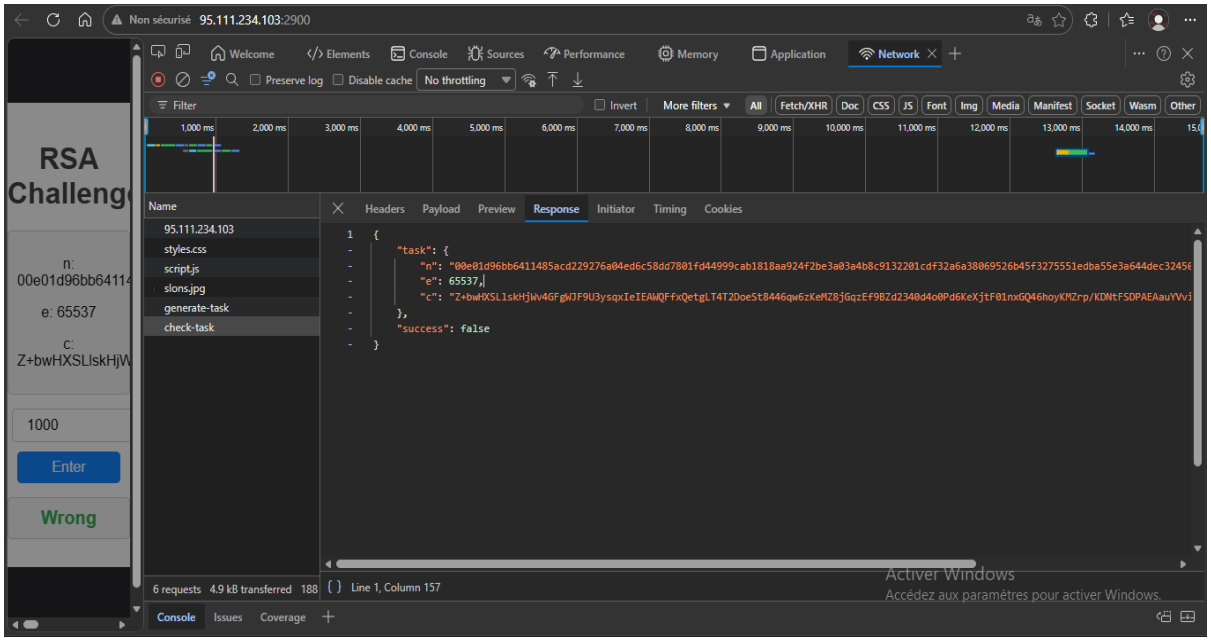
The challenge provided RSA parameters:

- **n (modulus)**
- **e (public exponent = 65537)**
- **c (ciphertext)**

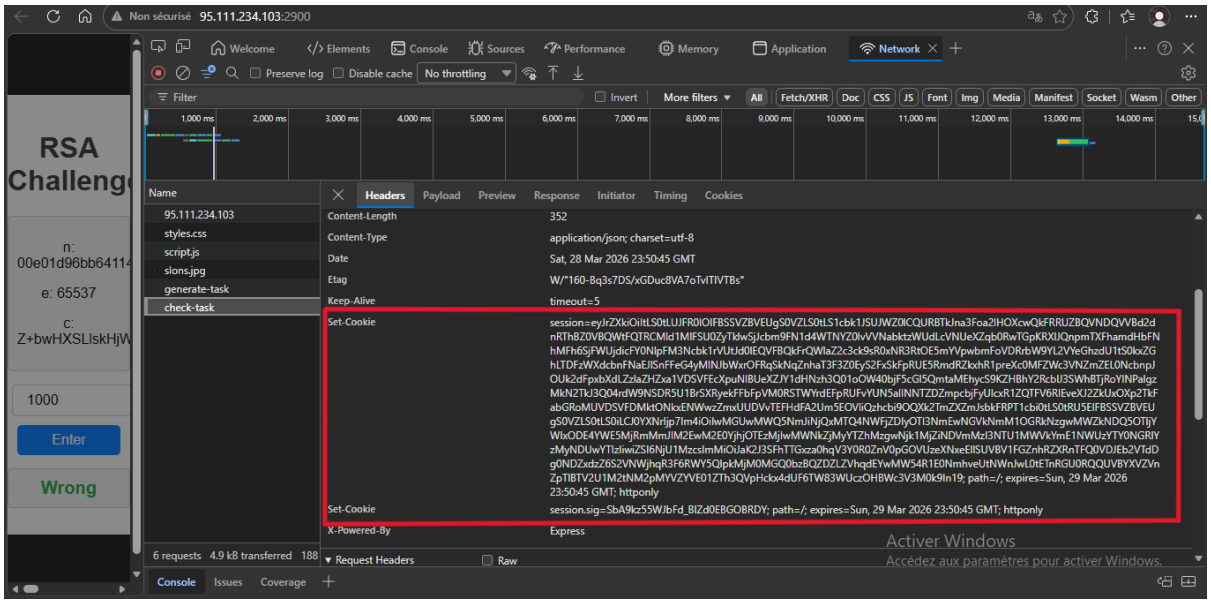
The goal was to recover the plaintext (flag)

Screenshot Evidence

- ### RSA Parameters Interface



- ### 2. DevTools – Network Leak



- request: `check-task`
- headers

- session cookie (encrypted)

Technical Analysis

From the challenge interface, I identified a standard RSA structure:

- Public key: (n, e)
- Ciphertext: c

The objective was to compute:

plaintext = c^d mod n

However, the private key **d** requires factorization of **n**.

What I Discovered

- The exponent **e = 65537** → standard secure RSA
- The modulus **n** seemed large → not directly factorable
- The ciphertext **c** was provided → typical encryption setup

From DevTools:

- I discovered a **session cookie leak**
- Observed backend interaction via `/check-task`
- The response system suggested server-side validation

Hypothesis (Very Important)

Based on the behavior, the challenge likely involved:

- Weak RSA parameters
- OR predictable key generation
- OR server-side validation bypass via session

Why I Didn't Solve It

- I did not complete factorization or exploit

- I lacked advanced number theory knowledge
- I did not implement a full Python script for RSA attack
- Time constraint during CTF

Technical Reflection

During this challenge, I moved beyond basic usage and started understanding how RSA works internally. I identified the relationship between public key components and ciphertext, and understood that breaking RSA requires exploiting weak parameters or implementation flaws.

I also discovered that sensitive data (like session cookies) can be exposed through browser DevTools, which may lead to alternative attack paths such as session manipulation or replay attacks.

Although I did not fully solve the challenge, I reached a critical point where the remaining step required deeper mathematical and scripting knowledge.

Intended Approach (Pseudo-Logic)

```
1. Extract n, e, c
2. Analyze size of n
3. Attempt factorization (if weak)
4. Compute private key d
5. Decrypt ciphertext
```

5. Dynamic Math Service (Failed)

Description

A server required solving 256 math problems in 21 seconds with changing operators.

Key Insight:

- Manual solving impossible
- Required full automation

Problem:

- High latency (ping)
- No complete script implemented

Learning:

- Speed matters
- Automation is mandatory in CTF

6. Seed Brute Force Challenge

Description

- Hidden seed between 1000 and 1,000,000
- Required brute force

Attempt:

- Understood brute-force concept
- Did not implement optimized script

Learning:

- CPU-based attacks require efficiency
- Need better Python performance skills

7. Skills Gained

- Encoding vs Encryption (clear understanding)
- RSA basics
- HTTP requests & headers
- Endpoint analysis
- DevTools usage for exploitation
- Python scripting fundamentals

8. Tools Used

- Python (automation)
- VPS (remote execution)
- DevTools (network analysis)
- Netcat (remote interaction)

9. Self-Assessment

Strengths:

- Logical thinking
- Web fundamentals
- Fast learning

Weaknesses:

- Cryptography depth
- Reverse Engineering
- Advanced scripting

10. Future Plan

- Study RSA and number theory
- Practice Python scripting daily
- Solve CTF challenges weekly
- Use platforms like Hack The Box / TryHackMe
- Build a GitHub portfolio

Conclusion

This CTF was a turning point in my learning.
Instead of only solving challenges, I started understanding the internal logic behind them.

Even without solving all tasks, I reached advanced stages in analysis, especially in RSA and automation-based challenges. This experience helped me identify my weaknesses and define a clear path for improvement.